

HDOC Day-28

NAME: Yug Nirwal

SAP ID 590033354

SUBJECT: C programming

DATE: 21-09-26

Question 1

Write a C program that accepts positive row and column values, creates a variable-length 2D integer array, reads the elements column by column, and prints them in row-major order using a single linear index.

Algorithm

1. Read rows repeatedly until a positive value is entered.
2. Read columns repeatedly until a positive value is entered.
3. Declare a variable-length array `a[rows][cols]`.
4. Use columns as the outer loop and rows as the inner loop to read elements column by column.
5. Use one loop from 0 to `rows*cols-1`. For each linear index `k`, obtain `row = k/cols` and `column = k%cols`, then print `a[row][column]`.

Test Cases

Test Case	Input	Expected Result
1	rows=2, cols=3; values entered column-wise: 1 2 3 4 5 6	1 3 5 2 4 6
2	rows=-2 then 2; cols=0 then 3; values: 10 20 30 40 50 60	10 30 50 20 40 60

C Program

```
#include <stdio.h>

int main() {
    int rows, cols;

    do {
        printf("Enter number of rows (positive): ");
        scanf("%d", &rows);
    } while (rows <= 0);

    do {
        printf("Enter number of columns (positive): ");
        scanf("%d", &cols);
    } while (cols <= 0);

    int a[rows][cols];

    printf("Enter %d elements column by column:\n", rows * cols);
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++) {
            scanf("%d", &a[i][j]);
        }
    }

    printf("Array in row-major order:\n");
    for (int k = 0; k < rows * cols; k++) {
        int i = k / cols;
        int j = k % cols;
        printf("%d ", a[i][j]);
    }
    printf("\n");
    return 0;
}
```

```
}
```

Execution / Test Case Output

Test Case 1

```
(kali~kali)-[~/Desktop/cprog]
└─$ ./q1
Enter number of rows (positive): 2
Enter number of columns (positive): 3
Enter 6 elements column by column:
1 2 3 4 5 6
Array in row-major order:
1 3 5 2 4 6
```

Test Case 2

```
(kali~kali)-[~/Desktop/cprog]
└─$ ./q1
Enter number of rows (positive): -2
Enter number of rows (positive): 2
Enter number of columns (positive): 0
Enter number of columns (positive): 3
Enter 6 elements column by column:
10 20 30 40 50 60
Array in row-major order:
10 30 50 20 40 60
```

Question 2

Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays.

Algorithm

6. Read positive values for rows and columns.
7. Declare a variable-length 2D array `a[rows][cols]` and a 1D array `b[rows*cols]`.
8. Read all 2D array elements row by row.
9. For every linear index `k`, copy `a[k/cols][k%cols]` into `b[k]`.
10. Display the 2D array row by row.
11. Display the 1D array in row-major order.

Test Cases

Test Case	Input	Expected Result
1	rows=2, cols=3; 1 2 3 4 5 6	2D: 1 2 3 / 4 5 6; 1D: 1 2 3 4 5 6
2	rows=3, cols=2; 10 20 30 40 50 60	2D: 10 20 / 30 40 / 50 60; 1D: 10 20 30 40 50 60

C Program

```
#include <stdio.h>

int main() {
    int rows, cols;

    do {
        printf("Enter number of rows (positive): ");
        scanf("%d", &rows);
    } while (rows <= 0);

    do {
        printf("Enter number of columns (positive): ");
        scanf("%d", &cols);
    } while (cols <= 0);

    int a[rows][cols];
    int b[rows * cols];

    printf("Enter %d elements row by row:\n", rows * cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    for (int k = 0; k < rows * cols; k++) {
        b[k] = a[k / cols][k % cols];
    }

    printf("2D array:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    printf("1D array in row-major order:\n");
    for (int k = 0; k < rows * cols; k++) {
        printf("%d ", b[k]);
    }
    printf("\n");
    return 0;
}
```

Execution / Test Case Output

Test Case 1

```
(kali~kali)-[~/Desktop/cprog]
└─$ ./q2
Enter number of rows (positive): 2
Enter number of columns (positive): 3
Enter 6 elements row by row:
1 2 3 4 5 6
2D array:
1 2 3
4 5 6
1D array in row-major order:
1 2 3 4 5 6
```

Test Case 2

```
(kali~kali)-[~/Desktop/cprog]
└─$ ./q2
Enter number of rows (positive): 3
Enter number of columns (positive): 2
Enter 6 elements row by row:
10 20 30 40 50 60
2D array:
10 20
30 40
50 60
1D array in row-major order:
10 20 30 40 50 60
```

Result

Both programs were compiled and tested successfully. The required variable-length arrays, traversal order, linear indexing, and row-major copying were verified using the prepared test cases.